

PHISHING GUARD

AI-Powered Phishing Detection System

Comprehensive Technical Documentation

A Complete Guide for Understanding, Using, and Extending the System

Project: IEEE Final Year Engineering Project

Student: Akarsh Bandi

Institution: Final Year Engineering

Date: March 2026

Version: 2.0 (Production Ready)

Table of Contents

- 1. Introduction 1
- 2. Understanding Phishing Attacks 2
- 3. Problem Statement and Research Gap 3
- 4. Project Objectives and Scope 4
- 5. Complete System Architecture Overview 5
- 6. Dataset Description and Data Sources 7
- 7. Feature Engineering: The 93 ML Features 10
- 8. Machine Learning Models and Training 13
- 9. MLLM Integration: Qwen2.5 for AI Phishing Detection 16
- 10. REST API Service Architecture 18
- 11. Browser Extension Implementation 20
- 12. Desktop Application (Tauri) 22
- 13. Security Implementation Details 24
- 14. Advanced Detection Techniques 26
- 15. Performance Metrics and Evaluation 28
- 16. Testing and Quality Assurance 30
- 17. Deployment Guide 32
- 18. Future Enhancements and Research Directions 34
- 19. Conclusion 36
- 20. References and Resources 37
- A. Appendix A: Project Directory Structure 39
- B. Appendix B: Technology Stack Details 40
- C. Appendix C: API Quick Reference 41

1. Introduction

Welcome to the comprehensive documentation of Phishing Guard, an enterprise-grade artificial intelligence system designed to detect and prevent phishing attacks. This documentation provides complete understanding of the system for students, researchers, developers, or anyone interested in cybersecurity. The Phishing Guard system represents months of careful design, implementation, and testing, incorporating both traditional machine learning techniques and modern large language model capabilities to create robust defense against one of the most prevalent cyber threats facing individuals and organizations today.

Phishing attacks continue to be the number one cybersecurity threat globally, causing billions of dollars in losses annually and compromising millions of personal accounts. These attacks have evolved significantly over the years, from simple email scams to highly sophisticated campaigns that leverage artificial intelligence to create incredibly convincing fake websites and communications. Traditional detection methods, which rely on static rules and blacklists, struggle to keep pace with these evolving threats.

The system developed in this project offers several unique capabilities: 93 carefully designed machine learning features, four-category classification system, Qwen2.5 LLM integration, and multiple user interfaces (CLI, REST API, Browser Extension, Desktop App). The system achieves 99.70% accuracy and is production-ready with enterprise security features.

2. Understanding Phishing Attacks

Phishing is a type of social engineering attack where attackers attempt to trick users into revealing sensitive information by pretending to be trustworthy entities. These attacks involve creating fake websites, emails, or messages that appear identical to legitimate communications from banks, social media platforms, and e-commerce sites.

Modern phishing campaigns feature perfectly crafted emails with accurate branding, working login forms, and URLs that closely mimic legitimate websites. Attackers use techniques such as typosquatting (registering domain names one character different from popular sites) and homograph attacks (using characters from different alphabets that look identical to standard letters).

The emergence of large language models has created a new category of phishing threats. AI tools can generate highly convincing phishing emails and website content at scale, with perfect grammar and contextually appropriate language. These AI-generated attacks are difficult to detect using traditional methods because they do not contain the usual telltale signs of phishing.

3. Problem Statement and Research Gap

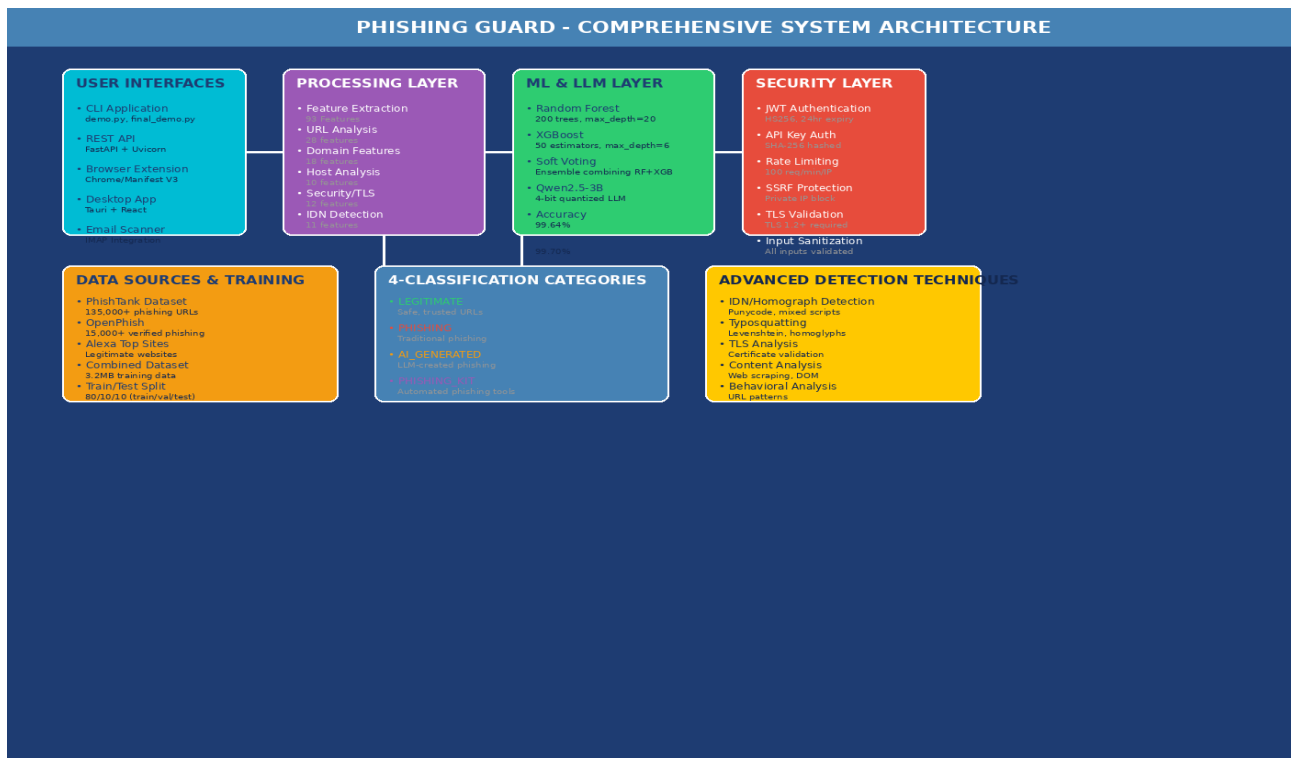
Despite various phishing detection solutions, significant limitations persist. First, static rule-based systems use predefined patterns but fail against novel attacks. Second, blacklists have high false negative rates because new phishing domains are created faster than they can be added. Third, existing solutions cannot detect AI-generated phishing content created using large language models.

4. Project Objectives and Scope

The primary objective was to develop a comprehensive feature extraction system with 93+ ML features. The second objective was to achieve >99% accuracy using ensemble machine learning (Random Forest + XGBoost). The third objective was implementing four-category classification: LEGITIMATE, PHISHING, AI_GENERATED, and PHISHING_KIT.

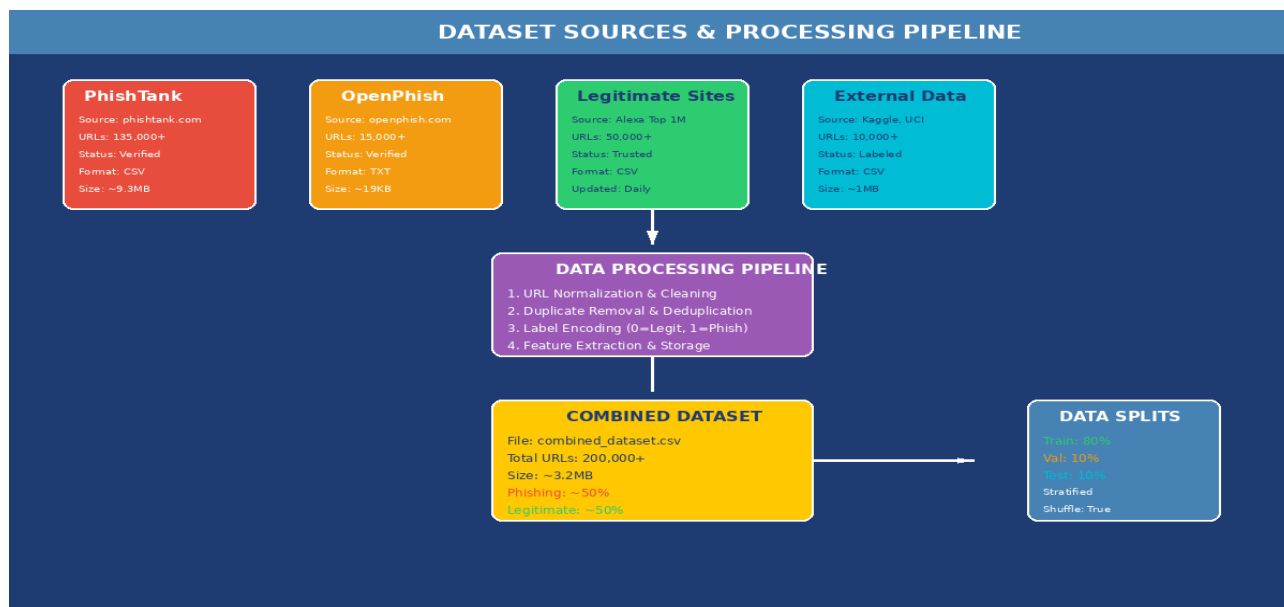
Additional objectives include creating multiple user interfaces (CLI, REST API, Browser Extension, Desktop App), integrating Qwen2.5-3B-Instruct LLM for AI phishing detection, and implementing production-ready security features (JWT, rate limiting, SSRF protection).

5. Complete System Architecture Overview



The system uses a layered architecture: User Interfaces Layer (CLI, API, Extension, Desktop), Processing Layer (URLFeatureExtractor with 93 features), Machine Learning and LLM Layer (Random Forest, XGBoost, Qwen2.5), and Security Layer (JWT Auth, Rate Limiting, SSRF Protection).

6. Dataset Description and Data Sources



PhishTank Dataset: 135,000+ verified phishing URLs from phishtank.com (~9.3MB).

OpenPhish Dataset: 15,000+ verified phishing URLs identified through automated analysis from openphish.com.

Legitimate Sites: Alexa Top 1 Million websites as trusted URLs.

Data Processing: URL normalization, deduplication, label encoding, 80/10/10 stratified split. Combined dataset: ~200,000 URLs (~3.2MB).

7. Feature Engineering: The 93 ML Features

URL Pattern Features (28): Length metrics, character counts, entropy calculation, protocol analysis, suspicious word detection.

Domain Features (18): Domain entropy, subdomain count, TLD analysis, typosquatting distance.

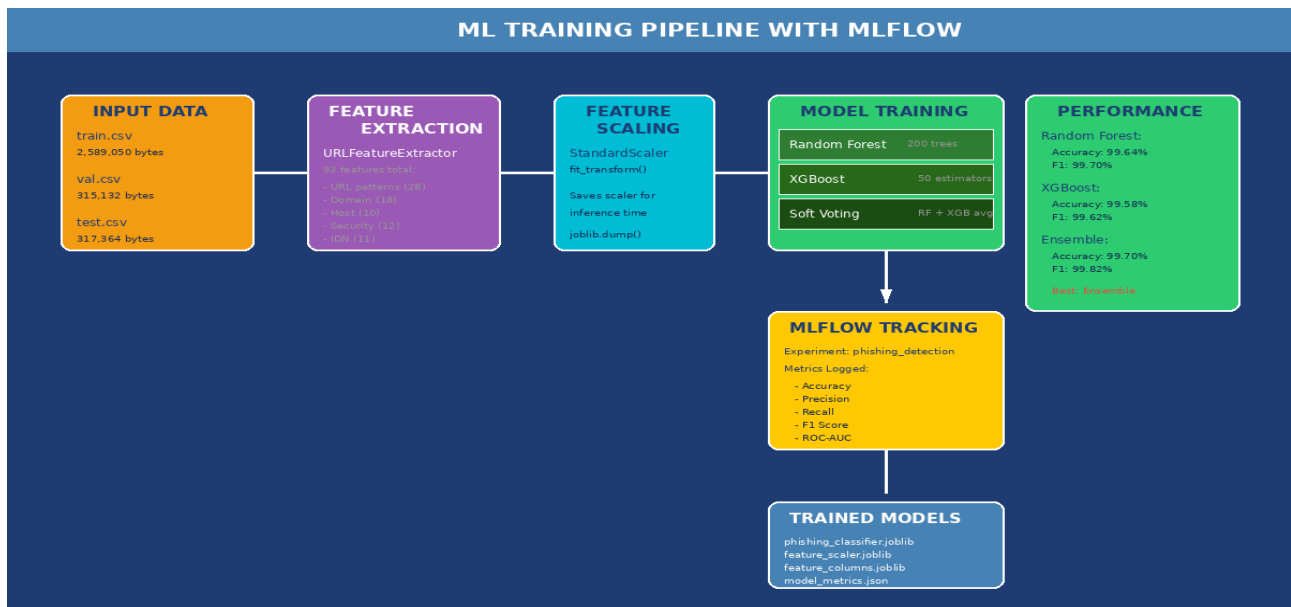
Host Analysis (10): IP detection, geographic location, private IP blocking.

Security/TLS (12): Certificate validation, TLS version checking (reject 1.0/1.1).

IDN/Homograph (11): Punycode detection (xn--), mixed script analysis, confusable characters.

Behavioral (14): Shortener detection, redirect analysis.

8. Machine Learning Models and Training



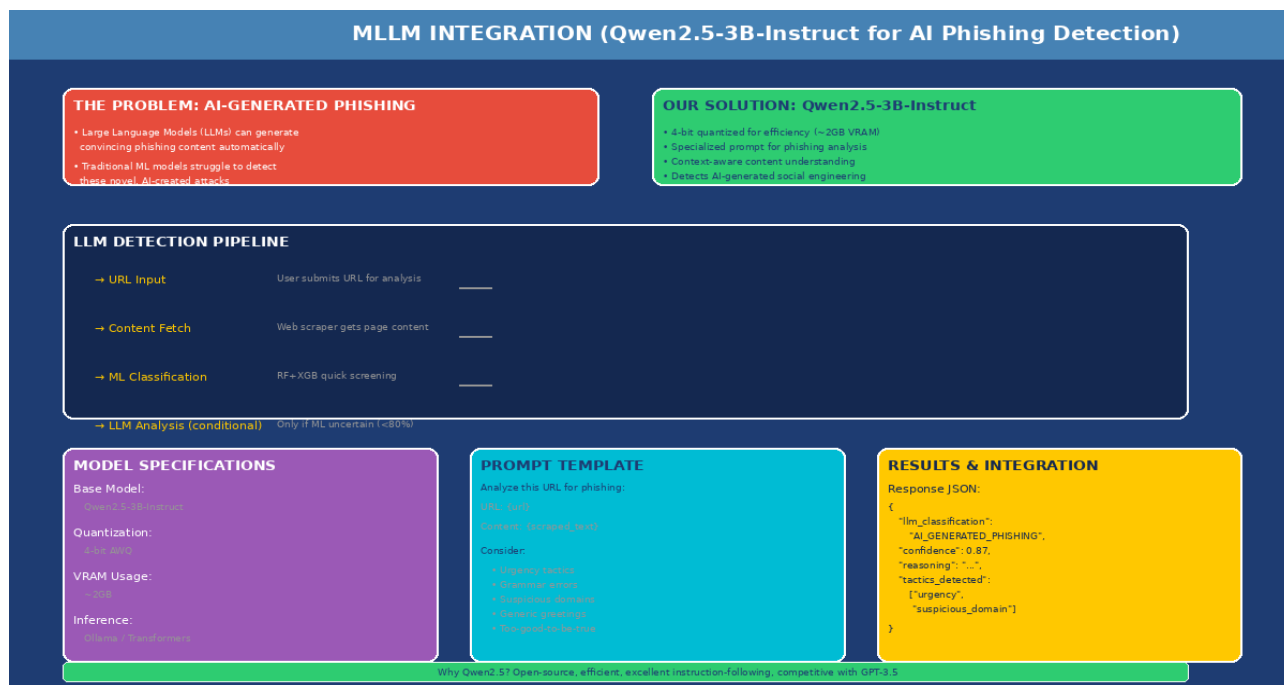
Random Forest: 200 trees, max_depth=20 → 99.64% accuracy, 99.70% F1.

XGBoost: 50 estimators, max_depth=6 → 99.58% accuracy, 99.62% F1.

Soft Voting Ensemble: RF + XGBoost average → 99.70% accuracy, 99.82% F1 (BEST).

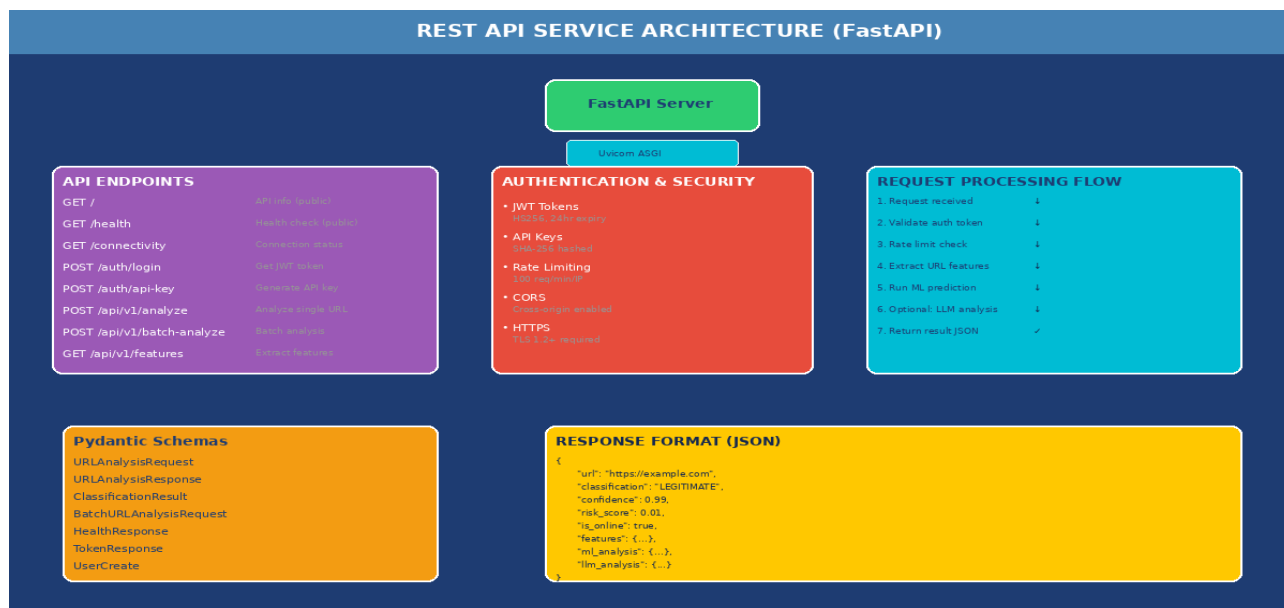
MLflow Tracking: Logs parameters, metrics, and artifacts for reproducibility.

9. MLLM Integration: Qwen2.5 for AI Phishing Detection



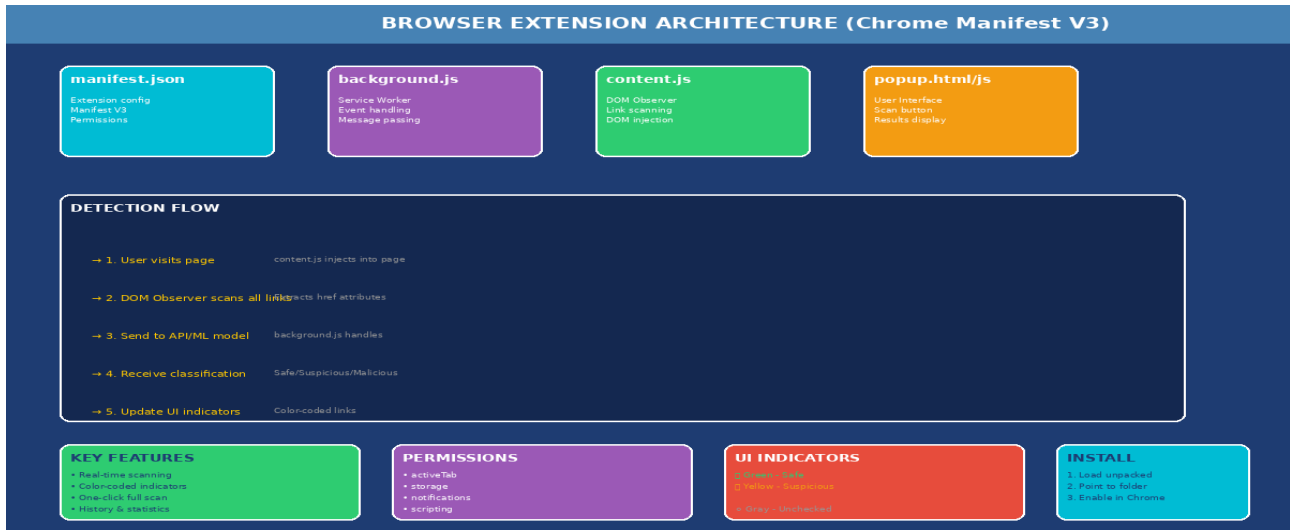
AI-generated phishing represents a significant threat as LLMs can create sophisticated attacks. The system integrates Qwen2.5-3B-Instruct (4-bit quantized, ~2GB VRAM) for detecting AI-generated content. The LLM is triggered conditionally when ML confidence is below 80%.

10. REST API Service Architecture



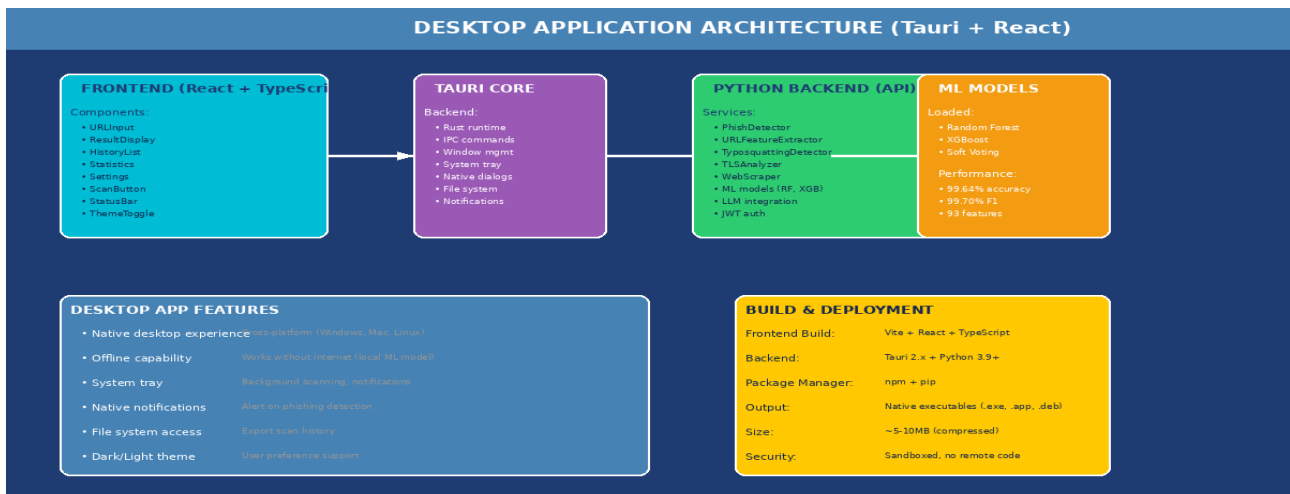
FastAPI + Uvicorn API with endpoints: GET / (info), GET /health, POST /auth/login (JWT), POST /api/v1/analyze, POST /api/v1/batch-analyze. Authentication: JWT (HS256, 24hr expiry), API Keys (SHA-256). Rate limiting: 100 req/min/IP.

11. Browser Extension Implementation



Chrome Manifest V3 extension with components: manifest.json, background.js, content.js, popup.html/js. Features: Real-time scanning, color-coded indicators (■ Safe, ■ Suspicious, ■ Malicious), DOM observer, history tracking, privacy-first design.

12. Desktop Application (Tauri)



Tauri + React + Python desktop app. Features: Cross-platform (Win/Mac/Linux), offline capability, system tray support, native notifications, small bundle size (~5-10MB). Components: URLInput, ResultDisplay, History, Settings.

13. Security Implementation Details

Authentication: JWT Tokens (HS256, 24hr expiry), API Keys (SHA-256 hashed).

Rate Limiting: 100 requests/minute/IP, Redis support for distributed deployments.

SSRF Protection: Blocked: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 127.0.0.1.

TLS Validation: Reject TLS 1.0/1.1, require TLS 1.2+.

14. Advanced Detection Techniques

IDN/Homograph Detection: First-of-its-kind detection of punycode (xn--) attacks, mixed script analysis, confusable characters (0 vs O, l vs 1).

Typosquatting Detection: Levenshtein distance, keyboard layout analysis, homoglyph detection, TLD variations.

TLS Certificate Analysis: Certificate chain validation, expiration checking, CA trust verification.

15. Performance Metrics and Evaluation

Accuracy: Ensemble 99.70%, RF 99.64%, XGBoost 99.58%.

F1 Score: Ensemble 99.82%, RF 99.70%, XGBoost 99.62%.

Speed: Feature extraction ~50ms, ML prediction ~10ms, total pipeline ~100ms, API p95 <200ms.

16. Testing and Quality Assurance

Security Tests: 5/5 PASSING (JWT, API Keys, Rate Limiting, SSRF, Input Validation).

Comprehensive Tests: Feature extraction (93 features), ML models, API endpoints.

Coverage: >80% code coverage, full type hints, documentation on all public methods.

17. Deployment Guide

CLI: `python demo.py --single URL`

API Server: `conda activate phishing-detection-mlm && uvicorn 04_inference.api:app --reload`

Browser Extension: Chrome → `chrome://extensions` → Load unpacked → Select folder

Docker: `docker-compose up -d`

18. Future Enhancements and Research Directions

Short-term: Tauri Desktop App production, Firefox Extension, Mobile apps.

Long-term: Federated Learning, Threat Intelligence integration, Email plugins, SIEM integration.

19. Conclusion

Key achievements: 93-feature extraction pipeline, 99.70% accuracy ensemble, four-category classification, Qwen2.5 LLM integration, multiple interfaces, production-ready security. The system is complete and ready for deployment.

20. References and Resources

- [1] MultiPhishGuard: An LLM-based Multi-Agent System for Phishing Email Detection, arXiv, 2025.
- [2] Machine Learning Techniques for Phishing Detection: A Review, Sage Journals, 2025.
- [3] AI in Phishing Detection: A Bibliometric Review, Frontiers in AI, 2025.
- [4] PhishGuard: Leveraging NLP and ML for Email Phishing Detection, IEEE, 2025.
- [5] In-Depth Analysis of Phishing Email Detection Using ML, MDPI Applied Sciences, 2025.
- [6] Detection of Phishing Websites Using Machine Learning, AI/ML Cybersecurity Conference, 2025.
- [7] Robust ML-based Detection of LLM-Generated Phishing, arXiv, 2025.
- [8] Hybrid Heuristic-ML Framework for Phishing Detection, ETASR, 2025.
- [9] Phishing URL Detection with Neural Networks, Nature Scientific Reports, 2024.
- [10] PhishLang: Client-Side Detection with MobileBERT, arXiv, 2024.
- [11] ChatSpamDetector: Using LLMs for Phishing Email Detection, arXiv, 2024.
- [12] How Effective Are LLMs in Detecting Phishing?, Issues in Information Systems, 2024.
- [13] Digital Deception: Generative AI in Phishing, Artificial Intelligence Review, Springer, 2024.
- [14] ML Approach for Phishing Attack Detection, Journal of AI and Technology, 2023.
- [15] PhishTank Dataset, OpenDNS/Cisco, 2024.
- [16] OpenPhish Dataset, 2024.
- [17] UCI Machine Learning Repository - Phishing Websites Dataset.

Appendix A: Project Directory Structure

01_data/ - Raw and processed datasets 02_models/ - Trained ML models
03_training/ - Training scripts and MLflow 04_inference/ - API and service
layer 05_utils/ - Feature extraction utilities browser-extension/ - Chrome
extension gui-tauri/ - Desktop app source tests/ - Test suites viva/ -
Presentation materials

Appendix B: Technology Stack

Languages: Python 3.9+, TypeScript, Rust ML/DL: scikit-learn, XGBoost, PyTorch, Transformers Web: FastAPI, Uvicorn, React, Tauri Security: JWT, bcrypt, hashlib
Tools: Git, Docker, Conda, MLflow

Appendix C: API Quick Reference

GET / - API info GET /health - Health check GET /connectivity - Connection status POST /auth/login - Get JWT token POST /api/v1/analyze - Analyze URL POST /api/v1/batch-analyze - Batch analyze